28

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 2

커스텀 데코레이터와 메타데이터

제2부 NestJS 핵심 · 6장 요청 파이프라인

코드에 표시를 붙이고, 그 표시대로 움직이게 하기



© 한상훈 NestJS 백엔드 마스터

취급주의 스티커

→ 상자 안: 그대로

→ 스티커를 읽은 기사: 맨 위에 싣고, 던지지 않음

→ 스티커를 아무도 안 읽으면: 아무 일도 없음



© 한상훈 NestJS 백엔드 마스터

지금까지 붙인 스티커

데코레이터	붙는 곳	읽는 쪽
`@Controller('concerts'), @Get(':id')`	클래스, 메서드	Nest 라우터
`@UseGuards, @UseInterceptors, @UseFilters`	클래스, 메서드	Nest 요청 처리
`@IsString(), @MaxLength(100)`	DTO 필드	ValidationPipe(class-validator)
`@Type(() => Number)`	DTO 필드	class-transformer
`@NotificationChannel('sms')`	프로바이더 클래스	알림 서비스(Discovery)

© 한상훈 NestJS 백엔드 마스터

판매 취소는 실장만

- → 매표소 판매: 판매원, 실장
- → 판매 취소(`release`): 실장만
- → 지금의 가드: 직원이면 통과, 기획팀만 403
- → 필요한 것: 핸들러마다 "누가 쓸 수 있나"를 적고 가드가 읽기
- → 데코레이터 = 함수: `@Roles(...)`는 클래스·메서드를 받아 값을 붙이고 끝남

© 한상훈 NestJS 백엔드 마스터

메타데이터

- → 뜻 — 클래스나 메서드에 붙여 두는 키와 값 (코드 실행에는 영향 없음)
- → 붙이는 쪽 — 데코레이터
- → 읽는 쪽 — `Reflector` (Nest가 주입해 주는 도구)
- → 예 — Roles 키에 `['manager']`를 release 메서드에

© 한상훈 NestJS 백엔드 마스터

Roles 데코레이터

roles.decorator.ts

```ts
import { Reflector } from '@nestjs/core';
import type { StaffRole } from '../staff/staff.service.js';

// 이 핸들러(또는 컨트롤러)를 쓸 수 있는 직원 역할. 붙어 있지 않으면 고객용
export const Roles = Reflector.createDecorator<StaffRole[]>();
```

→ 데코레이터와 읽는 키를 한 번에 생성

→ 문자열 키(`SetMetadata('roles', …)`): 'roles'와 'role' 오타 → 표시 없음으로 보고 통과

© 한상훈 NestJS 백엔드 마스터

표시를 읽는 StaffKeyGuard

staff-key.guard.ts

```ts
@Injectable()
export class StaffKeyGuard implements CanActivate {
  constructor(
    private readonly reflector: Reflector,
    private readonly staffService: StaffService,
  ) {}

  async canActivate(context: ExecutionContext): Promise<boolean> {
    const roles = this.reflector.getAllAndOverride(Roles, [context.getHandler(), context.getClass()]);
    if (!roles) return true; // 표시 없음: 고객용

    const req = context.switchToHttp().getRequest<Request & { staff?: Staff }>();
    const staff = await this.staffService.verify(req.header('x-staff-key'));
    if (!staff) throw new UnauthorizedException('직원 키가 없거나 맞지 않습니다');
    if (!roles.includes(staff.role)) throw new ForbiddenException(`${staff.name}님은 이 작업 권한이 없습니다`);

    req.staff = staff;
    return true;
  }
}
```

© 한상훈 NestJS 백엔드 마스터

판매와 판매 취소

box-office.controller.ts

```ts
@Controller('box-office/performances/:performanceId/seats')
@Roles(['clerk', 'manager'])
export class BoxOfficeController {
  @Post(':seatId/sell')
  sell(…) { … }

  // 판매 취소: 실장만
  @Post(':seatId/release')
  @Roles(['manager'])
  release(…) { return { ...this.seatsService.release(performanceId, seatId), releasedBy: staff.name }; }
}
```

→ `SeatsService.release`: SOLD → AVAILABLE, SOLD가 아니면 `SeatNotSoldError`(conflict)

© 한상훈 NestJS 백엔드 마스터

세 직원으로 확인

curl

```
판매 김매표       {"id":"A-1-1", …,"status":"SOLD","soldBy":"김매표"} [201]
취소 김매표       {"title":"Forbidden","status":403,"detail":"김매표님은 이 작업 권한이 없습니다", …} [403]
취소 이실장       {"id":"A-1-1", …,"status":"AVAILABLE","releasedBy":"이실장"} [201]
다시 취소 이실장  {"seatId":"A-1-1","type":"…/problems/seat-not-sold","status":409, …} [409]
판매 박기획       {"title":"Forbidden","status":403,"detail":"박기획님은 이 작업 권한이 없습니다", …} [403]
```

© 한상훈 NestJS 백엔드 마스터

메서드만 읽은 가드

```
const roles = this.reflector.get(Roles, context.getHandler()); // 메서드만 읽음
if (!roles) return true; // 표시 없음: 고객용
```

→ `sell` 메서드: Roles 없음 (클래스에만 있음)

→ `release` 메서드: `Roles(['manager'])` 있음

© 한상훈 NestJS 백엔드 마스터

응답은 500, 좌석은 SOLD

curl · 서버 로그

```
$ curl -X POST localhost:4028/box-office/performances/1/seats/A-1-1/sell      (키 없음)
{"type":"about:blank","title":"Internal Server Error","status":500,
 "detail":"요청을 처리하지 못했습니다.", …} [500]
ERROR [ProblemDetails] TypeError: Cannot read properties of undefined (reading 'name')

$ curl "localhost:4028/performances/1/seats?row=1&status=SOLD"
[{"id":"A-1-1","label":"A구역 1열 1번","status":"SOLD"}]

$ curl -X POST localhost:4028/box-office/performances/1/seats/A-1-1/release   (키 없음)
{"title":"Unauthorized","status":401, …} [401]
```

→ 가드 통과 → 좌석 판매 실행 → `staff.name`에서 에러

© 한상훈 NestJS 백엔드 마스터

getAllAndOverride

읽는 방법	결과
`get(Roles, handler)`	메서드 표시만, 클래스 표시는 못 봄
`getAllAndOverride(Roles, [handler, class])`	메서드 표시가 있으면 그것, 없으면 클래스 표시
`getAllAndMerge(Roles, [handler, class])`	둘을 합침 (배열은 이어 붙임)

→ release: 메서드 `['manager']`가 클래스 `['clerk','manager']`를 덮음

→ 시험: 키 없이 매표소 주소를 전부 호출해 하나도 열리지 않는지부터

© 한상훈 NestJS 백엔드 마스터

전역 가드와 표시

```
// AppModule
imports: [ /* … */ StaffModule],
providers: [
  AppService,
  { provide: APP_GUARD, useClass: StaffKeyGuard },
  // …
],
```

○ ○ ○ curl

```
$ curl localhost:4028/concerts
[200]
```

→ BoxOfficeController의 `@UseGuards` 삭제, SeatsModule의 StaffModule import 삭제

→ Roles가 붙은 컨트롤러의 새 메서드: 자동으로 직원 확인

→ 고객 API에서 가드 비용: 표시 조회 한 번 뒤 통과, 직원 조회(DB)는 표시가 있을 때만

© 한상훈 NestJS 백엔드 마스터

기본 허용과 기본 거부

방식	표시가 없으면	깜빡했을 때
기본 허용 (지금)	고객용, 통과	직원 기능이 열림
기본 거부	막음, 공개하려면 `@Public()` 같은 표시	고객 기능이 막힘

→ 지금 고른 이유: 고객 API가 대부분, 직원 API는 매표소 컨트롤러 하나

→ 깜빡했을 때 결과: 기본 거부 쪽이 덜 위험

© 한상훈 NestJS 백엔드 마스터

CurrentStaff

current-staff.decorator.ts

```ts
import { createParamDecorator, ExecutionContext } from '@nestjs/common';
import type { Staff } from '../staff/staff.service.js';

// StaffKeyGuard가 req.staff에 실어 둔 직원을 핸들러 인자로
export const CurrentStaff = createParamDecorator(
  (_data: unknown, ctx: ExecutionContext): Staff => ctx.switchToHttp().getRequest().staff,
);
```

© 한상훈 NestJS 백엔드 마스터

핸들러에서 쓰기

box-office.controller.ts

```
@Post(':seatId/sell')
sell(
  @Param('performanceId', ParseIntPipe) performanceId: number,
  @Param('seatId', SeatIdPipe) seatId: string,
  @CurrentStaff() staff: Staff,
) {
  return { ...this.seatsService.sell(performanceId, seatId), soldBy: staff.name };
}
```

→ `@Req()`와 `Request & { staff }` 타입 우기기가 사라짐

→ 같은 방식: `@Param`, `@Body`, `@Query`도 파라미터 데코레이터

© 한상훈 NestJS 백엔드 마스터

applyDecorators로 묶기

```ts
// 직원 전용 표시 + 응답을 어디에도 저장하지 말 것
export const StaffOnly = (...roles: StaffRole[]) =>
  applyDecorators(Roles(roles), Header('Cache-Control', 'no-store'));
```

curl

```
$ curl -i -X POST …/seats/A-1-1/sell -H 'x-staff-key: key-kim'
HTTP/1.1 201 Created
Cache-Control: no-store
```

→ 판매·취소 메서드에 `@StaffOnly(…)` (시연 가지)

→ 클래스에 붙였을 때: 부팅 중 TypeError (`@Header`는 메서드 전용)

데이터 블록: @Header를 클래스에 붙였을 때: 부팅 중 TypeError (@nestjs/common 12.2.0 실측, 2026-10-26)

© 한상훈 NestJS 백엔드 마스터

@Exclude와 ClassSerializerInterceptor

concert.entity.ts

```
export class Concert {
  id: number;
  title: string;
  venue: string;

  @Exclude()
  settlementRate: number;

  createdAt: Date;
}
```

main.ts

```
// main.ts
app.useGlobalInterceptors(new ClassSerializerInterceptor(app.get(Reflector)));
// 시연: 공연 조회가 응답 DTO 대신 엔티티를 그대로 돌려줌
```

© 한상훈 NestJS 백엔드 마스터

객체 리터럴과 인스턴스

curl

```
(ConcertsService.create가 객체 리터럴로 만든 코드)
$ curl localhost:4028/concerts/1
{"id":1,"title":"가을 콘서트","venue":"올림픽홀","settlementRate":0.1,
 "createdAt":"2026-10-26T06:28:02.019Z"}

(Object.assign(new Concert(), { … })로 만든 코드)
$ curl localhost:4028/concerts/1
{"id":1,"title":"가을 콘서트","venue":"올림픽홀",
 "createdAt":"2026-10-26T06:28:07.360Z"}
```

→ `@Exclude`: Concert 클래스에 붙은 표시

→ 객체 리터럴: Concert 인스턴스가 아님 → 읽을 표시가 없음

→ 인스턴스 만들기: `new Concert()` + `Object.assign`, 또는 `plainToInstance`

© 한상훈 NestJS 백엔드 마스터

감추는 두 방법

방법	감추는 기준	새는 경우
응답 DTO로 옮기기 (지금 프로젝트)	응답 DTO에 적은 필드만	응답 DTO 변환을 빠뜨린 핸들러
`@Exclude` + ClassSerializerInterceptor	엔티티에 붙은 표시	인스턴스가 아닌 객체를 돌려준 핸들러

"요청 DTO에는 받을 것만, 응답 DTO에는 보여 줄 것만 적고, 엔티티는 밖으로 그대로 내보내지 않습니다."

© 한상훈 NestJS 백엔드 마스터

max-age=undefined

```
@Get()
@Header('Cache-Control', `max-age=${process.env.SEAT_MAP_MAX_AGE}`)
findAll(…) { … }
```

○ ○ ○ 서버 로그 · curl

```
.env: SEAT_MAP_MAX_AGE=2

bootstrap 시점 SEAT_MAP_MAX_AGE = 2
$ curl -i "localhost:4028/performances/1/seats?row=1"
Cache-Control: max-age=undefined
```

© 한상훈 NestJS 백엔드 마스터

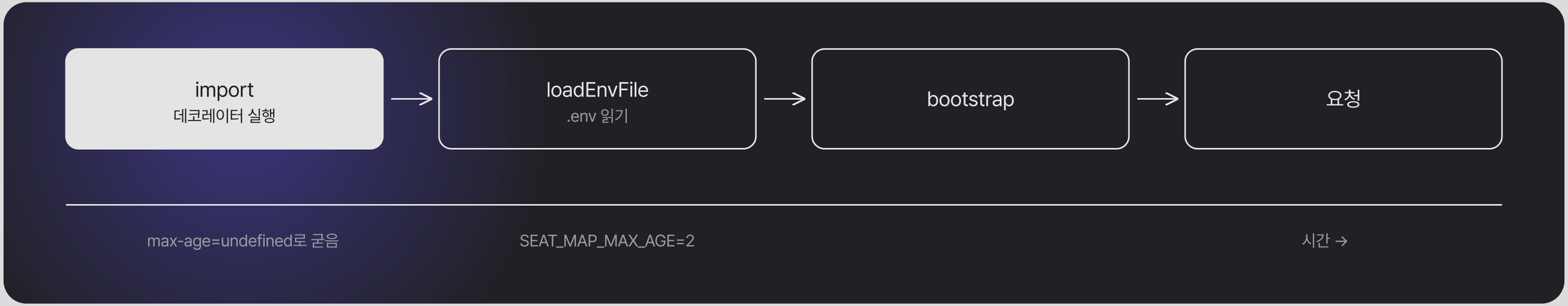
import가 먼저

→ main.ts의 import 줄: 모듈 파일 전부를 먼저 불러옴

→ 불러오는 순간: 클래스 정의, 데코레이터 실행, 인자 계산 (한 번)

→ 그다음 main.ts 본문: `process.loadEnvFile()` → .env 읽기

→ 데코레이터가 본 `process.env.SEAT_MAP_MAX_AGE`: 아직 없음



© 한상훈 NestJS 백엔드 마스터

표시와 읽는 쪽

표시 (데코레이터)	읽는 쪽	읽는 시점
`@Roles([...])`	StaffKeyGuard (Reflector)	요청마다
`@CurrentStaff()`	Nest 인자 처리	요청마다
`@Exclude()`	ClassSerializerInterceptor	응답마다
`@NotificationChannel('sms')`	알림 서비스 (Discovery)	부팅 때 한 번

→ 같은 구조: `DiscoveryService.createDecorator(@NotificationChannel)`, `Reflector.createDecorator(@Roles)`

→ 데코레이터 인자: 상수로

→ 바뀌는 값(환경 변수, 설정): 읽는 쪽에서 요청 때 또는 부팅 뒤에

© 한상훈 NestJS 백엔드 마스터

커스텀 데코레이터와 메타데이터

"데코레이터는 클래스에 표시만 남기고, 그 표시를 읽고 움직이는 건 가드와 인터셉터입니다."